

Experiment No.	4
Experiment Title	Binary Classification using Linear and Kernel-Based Models 1
Student Name	Raghu
Register Number	
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 18, 2026

1 Objective

To classify emails as spam or ham using Logistic Regression and Support Vector Machine (SVM) classifiers, and to analyze the effect of hyperparameter tuning (regularization strength, solver, kernel choice, C , and γ) on classification performance.

2 Problem Statement

Given a set of 57 numerical features derived from email content (word frequencies, character frequencies, and capital-letter run-length statistics), predict a binary label indicating whether an email is *spam* (1) or *ham* (0). The input is a 4601×57 feature matrix; the output is a binary class prediction. The objective is to build, tune, and compare a linear probabilistic classifier (Logistic Regression) against a margin-based kernel classifier (SVM) on this task.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	UCI Machine Learning Repository / Kaggle (somesesh24/spambase)
Number of Samples	4601 raw (4210 after duplicate removal)
Number of Features	57 (48 word-frequency, 6 character-frequency, 3 capital-run-length)
Number of Classes	2 (Spam = 1, Ham = 0)
Missing Values	0
Train-Test Split	80% / 20%, stratified (3368 train / 842 test)

4 Exploratory Data Analysis

Class Distribution

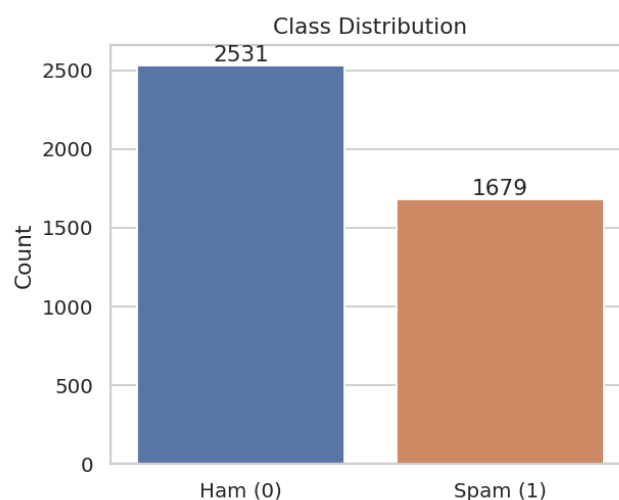


Figure 1: Distribution of Ham vs. Spam emails in the dataset (after duplicate removal: 2531 ham, 1679 spam).

Inference: The dataset is moderately imbalanced, with ham emails outnumbering spam by roughly a 3:2 ratio. This is not severe enough to require resampling techniques such as SMOTE, but it does justify reporting precision, recall, and F1-score alongside accuracy rather than relying on accuracy alone. Stratified splitting was used during train-test division to preserve this ratio in both subsets. The imbalance also motivates monitoring recall on the spam class specifically, since missed spam (false negatives) and misclassified ham (false positives) carry different practical costs for an email filter.

Statistical Summary

A full statistical summary (mean, standard deviation, quartiles) for all 57 features was computed and exported to `results/statistical_summary.csv`. Word-frequency and character-frequency features are heavily right-skewed (most emails have a frequency of 0 for any given word), while the capital-run-length features span several orders of magnitude, motivating standardization before training either model.

Missing Value Analysis

The dataset contains **zero missing values** across all 4601 rows and 58 columns, so no imputation was required. **391 duplicate rows** were found and removed prior to splitting, to prevent the same email's feature vector from appearing in both the training and test partitions.

Correlation Matrix

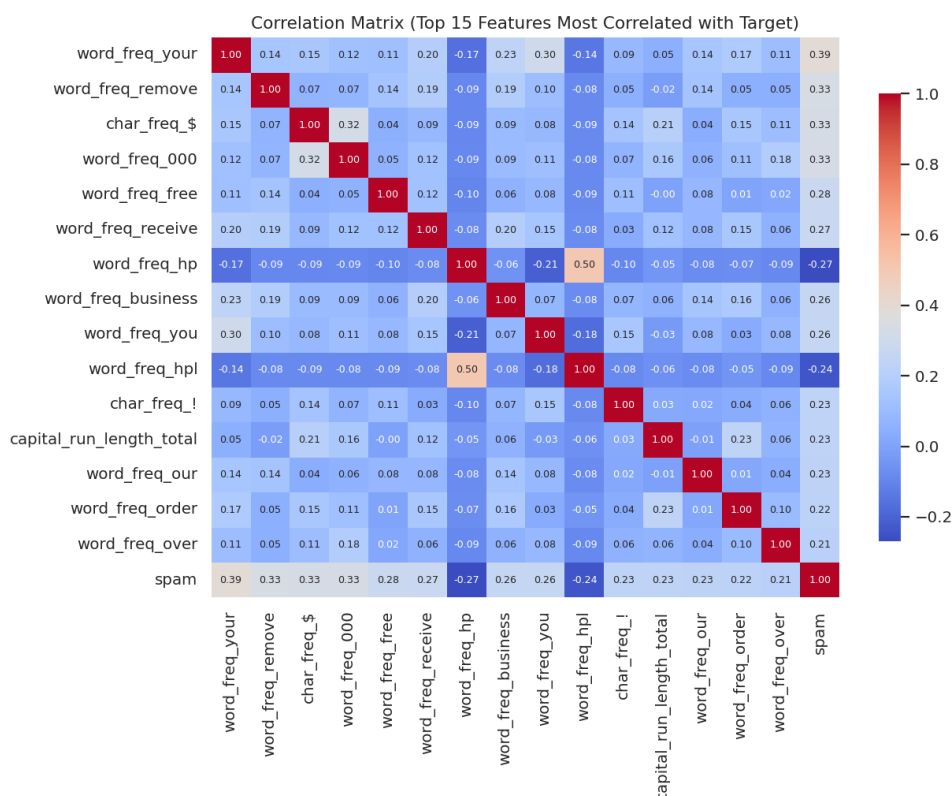


Figure 2: Correlation heatmap of the 15 features most correlated with the target label spam.

Inference: Character frequency of ! and \$, along with word frequencies for terms like “remove”, “free”, and “your”, show the strongest positive correlation with the spam label, matching intuitive expectations about promotional/spam language. Most pairwise feature correlations are weak-to-moderate, suggesting limited multicollinearity, though a few word-frequency features (e.g. “650”, “technology”, “hp”, “george”) are correlated with each other since they co-occur in the same class of corporate emails in the original HP Labs corpus.

Feature Distribution by Class

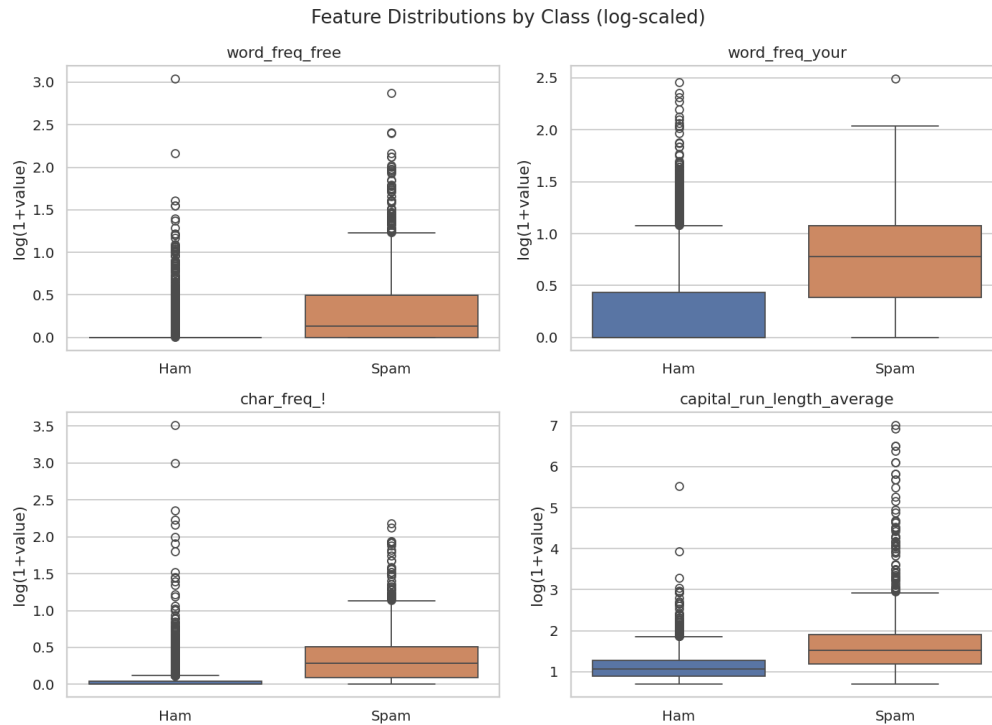


Figure 3: Log-scaled distributions of four representative features, split by class.

Inference: Spam emails show consistently higher values for `word_freq_free`, `word_freq_your`, `char_freq_!`, and `capital_run_length_average` compared to ham emails, with visibly higher medians and wider upper-quartile spread in the spam class. This confirms these features carry strong discriminative signal individually, even before being combined by a classifier, and explains why linear decision boundaries already perform well on this dataset.

5 Data Preprocessing

- **Missing value handling:** None required — the dataset had zero missing entries.
- **Duplicate handling:** 391 duplicate rows were dropped (4601 $\rightarrow$ 4210 rows) to avoid train/test leakage.
- **Encoding:** Not required — all 57 features are already numeric (frequencies and counts); the target is already binary.
- **Feature scaling:** `StandardScaler` (zero mean, unit variance) was fit on the training set only and applied to both train and test sets, since both Logistic Regression (regularization penalty) and SVM (distance-based margin, RBF kernel) are scale-sensitive.
- **Train-test split:** 80%/20% stratified split (`random.state=42`) to preserve the class ratio in both partitions.

- **Feature engineering:** Not applied — the 57 provided features were used as-is; the dataset dictionary already encodes domain-specific frequency features.

6 Machine Learning Algorithm

Logistic Regression

Working principle: Logistic Regression models the log-odds of the positive class as a linear combination of input features, mapped through the sigmoid function to a probability:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

A threshold (0.5 by default) converts this probability into a class label. Parameters w, b are learned by minimizing the log-loss (cross-entropy), optionally with an L1 or L2 penalty on w .

Advantages: Fast to train, naturally probabilistic output, highly interpretable coefficients, works well when classes are near-linearly separable, and L1 regularization gives built-in feature selection.

Limitations: Assumes a linear decision boundary in feature space; struggles with complex, non-linear class boundaries unless features are manually engineered or transformed.

Support Vector Machine (SVM)

Working principle: SVM finds the hyperplane that maximizes the margin between the two classes. Only the closest points (support vectors) determine the boundary. The kernel trick implicitly maps inputs into a higher-dimensional space, allowing non-linear boundaries:

$$f(x) = \text{sign} \left(\sum_i \alpha_i y_i K(x_i, x) + b \right)$$

where K is the kernel function (linear, polynomial, RBF, or sigmoid in this experiment).

Advantages: Effective in high-dimensional spaces, robust to overfitting when the margin is well-regularized, and kernels allow flexible non-linear boundaries without explicit feature transformation.

Limitations: Training and hyperparameter search scale poorly with dataset size, less interpretable than Logistic Regression, and performance is sensitive to the choice of kernel and C/γ values.

7 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.8.0
IDE	Claude Code sandbox (headless)
Operating System	Ubuntu 24.04 (Linux container)
Hardware	CPU-only container

8 Hyperparameter Selection

Parameter	Search Space
LR: Regularization (penalty)	L1, L2
LR: C	0.01, 0.1, 1, 10, 100
LR: Solver	liblinear, saga
SVM: Kernel	Linear, Polynomial, RBF, Sigmoid
SVM: C	0.1, 1, 10, 100
SVM: γ	scale, auto
SVM: Degree (Polynomial)	2, 3, 4

Logistic Regression was tuned with **Grid Search** (exhaustive, 20 combinations $\times$ 5-fold CV = 100 fits) since its search space is small. SVM was tuned with **Randomized Search** (20 sampled combinations $\times$ 5-fold CV = 100 fits) since its combined kernel/ C / γ /degree space is much larger and grid search would be computationally expensive, per the note in the lab manual permitting either method.

9 Results

Hyperparameter Tuning Results

Model	Search Method	Best Parameters	Best CV Accuracy
Logistic Regression	Grid Search	$C=1$, penalty=L1, solver=liblinear	0.9243
SVM	Randomized Search	kernel=RBF, γ =scale $C=10$,	0.9255

Logistic Regression Performance (Tuned, on held-out test set)

Metric	Value
Accuracy	0.9371
Precision	0.9301
Recall	0.9107
F1 Score	0.9203
ROC-AUC	0.9774
Training Time (single fit, s)	0.091

SVM Kernel-wise Performance (baseline, default hyperparameters)

Kernel	Accuracy	F1 Score	Training Time (s)
Linear	0.9359	0.9189	0.34
Polynomial	0.7660	0.5988	0.35
RBF	0.9323	0.9135	0.18
Sigmoid	0.8967	0.8676	0.21

Note: After tuning, the best SVM kernel was RBF with $C = 10$ (test accuracy 0.9335, F1 0.9154, ROC-AUC 0.9776) — a modest improvement over its untuned default ($C = 1$) accuracy of 0.9323, and better than the baseline Polynomial kernel by a wide margin.

K-Fold Cross-Validation Results ($K = 5$, on training set, tuned models)

Fold	Logistic Regression	SVM
Fold 1	0.9125	0.9214
Fold 2	0.9332	0.9421
Fold 3	0.9125	0.8976
Fold 4	0.9198	0.9406
Fold 5	0.9346	0.9406
Average	0.9225 (± 0.0097)	0.9285 (± 0.0172)

10 Comparative Analysis

Criterion	Logistic Regression	SVM
Accuracy (test set)	0.9371	0.9335
Model Complexity	Low	High
Training Time	Low (0.09 s)	Higher (0.20 s single fit; search itself costlier)
Interpretability	High	Low

Observations

- **Best-performing classifier:** Tuned Logistic Regression edges out tuned SVM on the held-out test set (93.71% vs. 93.35% accuracy, F1 0.9203 vs. 0.9154), though the two are within about half a percentage point of each other and their ROC-AUC scores are statistically indistinguishable (0.9774 vs. 0.9776). In 5-fold cross-validation on the training set, SVM's average accuracy (0.9285) is actually slightly higher than Logistic Regression's (0.9225), but with almost double the fold-to-fold standard deviation (0.0172 vs. 0.0097) — Logistic Regression is the more *consistent* performer across folds, even where SVM's mean is marginally higher.
- **Impact of regularization:** Grid search selected **L1 (Lasso)** regularization with a moderate $C = 1$ for Logistic Regression, indicating that a sparse subset of the 57 features carries most of the discriminative signal and that overly weak regularization (large C) is not needed given the data volume. The L1 penalty also aids interpretability, as it drives many weakly-informative word-frequency coefficients toward zero.
- **Kernel behavior in SVM:** Linear and RBF kernels perform almost identically and substantially outperform Sigmoid and especially Polynomial, whose default-hyperparameter recall collapses to 0.44 (it labels most spam as ham). This strongly suggests the two classes are close to linearly separable in the standardized 57-dimensional feature space — consistent with Logistic Regression's strong linear

performance — while the Polynomial kernel overfits/underfits badly without careful degree and γ tuning.

- **Bias-variance trade-off:** Logistic Regression, as a linear, high-bias/low-variance model, generalizes very consistently (low CV standard deviation) despite its simplicity, since the underlying data is close to linearly separable. The tuned RBF-kernel SVM, with more capacity (lower bias, higher variance), fits the training folds more flexibly — reflected in a higher CV mean but larger CV spread — while its held-out test performance lands marginally below Logistic Regression, a mild sign that its added flexibility is not being rewarded on this dataset.

11 Result Visualization

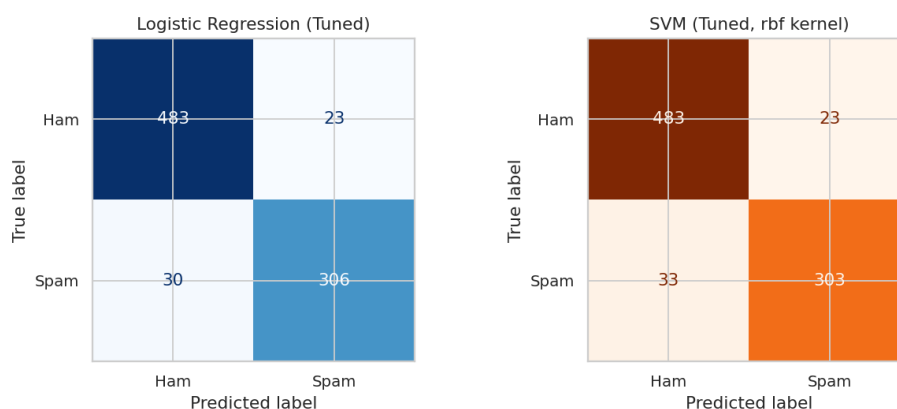


Figure 4: Confusion matrices for tuned Logistic Regression (left) and tuned SVM/RBF (right) on the test set.

Inference: Both models make a similar number of errors overall (~ 53 – 56 out of 842), concentrated slightly more on false negatives (spam predicted as ham) than false positives. This is expected given the moderate class imbalance and the 0.5 decision threshold; a stricter spam filter in production might shift this threshold to reduce false positives (legitimate mail marked as spam) at some cost to recall.

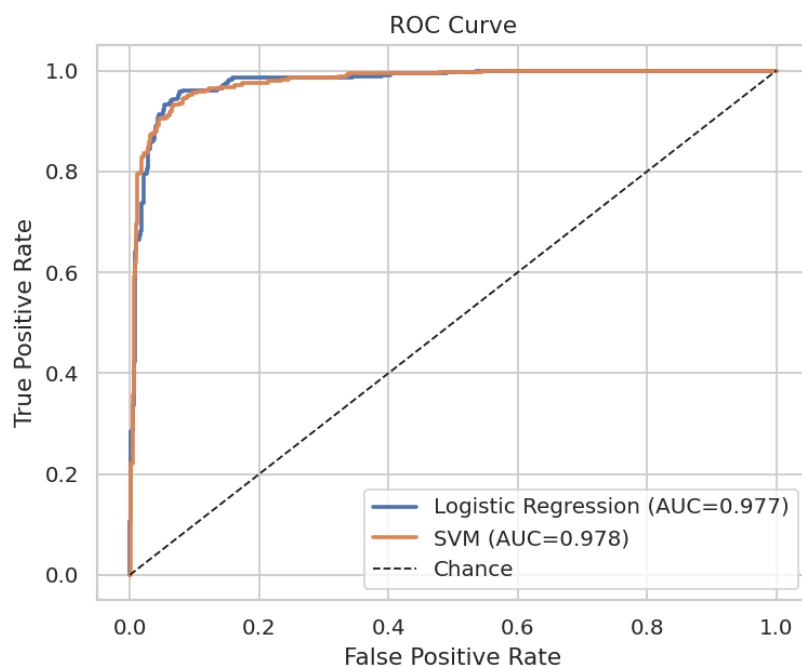


Figure 5: ROC curves for tuned Logistic Regression and tuned SVM.

Inference: Both curves rise steeply toward the top-left corner and are nearly indistinguishable, with ROC-AUC of 0.9774 (LR) and 0.9776 (SVM). This confirms both classifiers rank spam probability almost equally well across all thresholds, even though their default-threshold accuracy differs slightly — the AUC gap is well within noise.

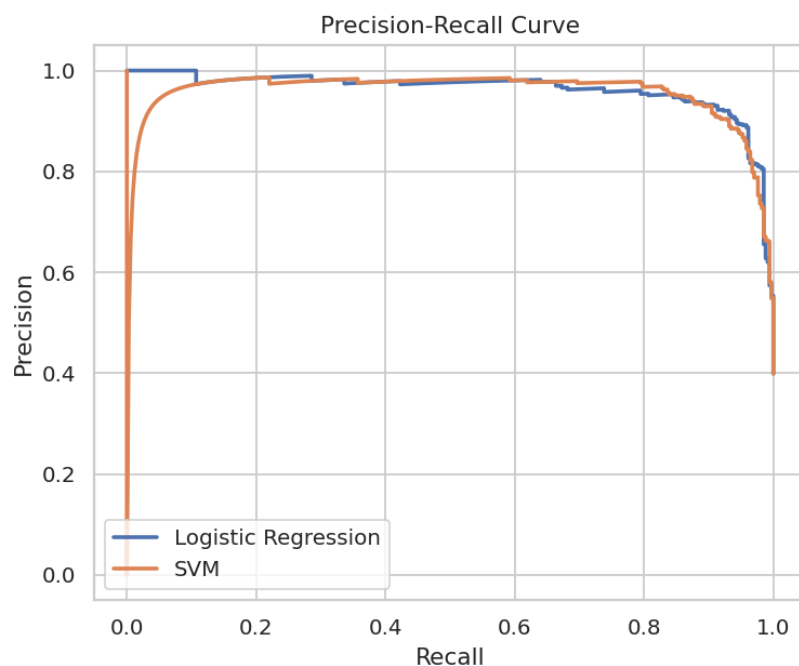


Figure 6: Precision-Recall curves for tuned Logistic Regression and tuned SVM.

Inference: Precision stays above 0.9 for recall values up to roughly 0.85 for both

models, then drops off as recall approaches 1.0 — i.e., correctly catching the last few, hardest-to-detect spam emails costs a disproportionate amount of precision (more false positives). The two curves overlap closely, reinforcing that neither model has a clear practical advantage on this dataset.



Figure 7: Learning curve for tuned Logistic Regression.

Inference: The training and cross-validation curves converge closely as training size increases, with a small and shrinking gap by the time the full training set is used. This indicates low variance/overfitting for the tuned Logistic Regression model, and suggests that collecting substantially more training data would yield only marginal further accuracy gains on this feature set.

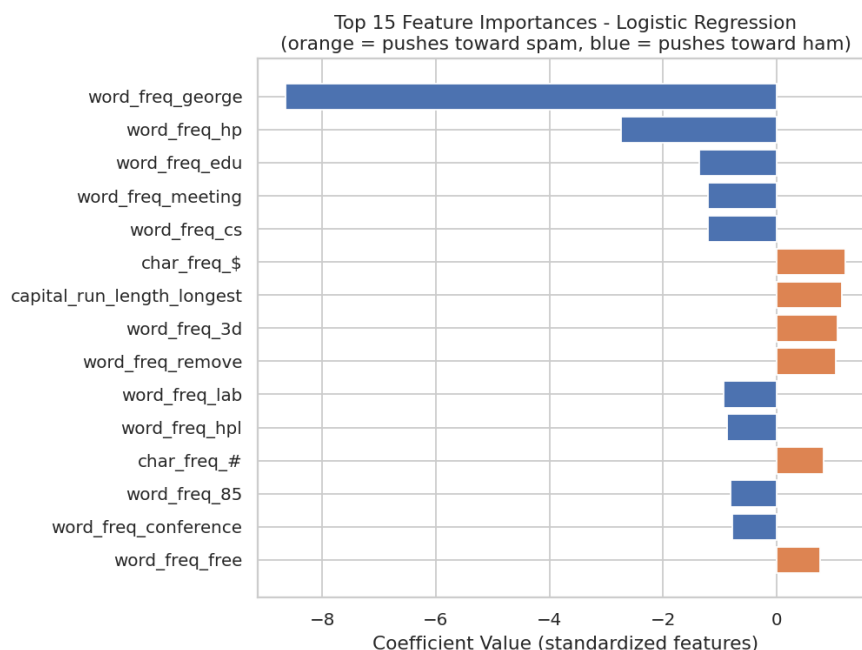


Figure 8: Top 15 feature coefficients (by magnitude) from tuned Logistic Regression.

Inference: Features such as `char_freq_$`, `word_freq_remove`, `word_freq_free`, and `word_freq_000` carry the largest positive coefficients (push predictions toward spam), while features like `word_freq_hp`, `word_freq_george`, and `word_freq_edu` carry large negative coefficients (push toward ham) — consistent with the original HP Labs corpus containing internal corporate/academic ham email alongside promotional spam. This matches domain intuition and confirms the model has learned semantically sensible decision rules rather than spurious correlations.

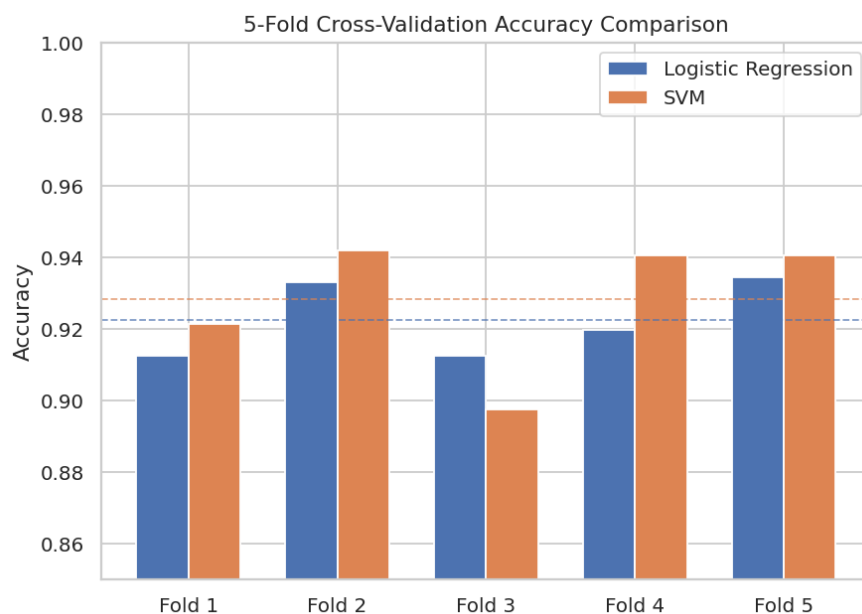


Figure 9: Per-fold 5-fold cross-validation accuracy comparison.

Inference: SVM’s per-fold accuracy is more volatile (dipping to 0.898 on Fold 3 while peaking at 0.942 on two other folds) than Logistic Regression’s, which stays in a tighter band (0.912–0.935). This visually reinforces the earlier bias-variance observation: Logistic Regression is the more stable model across resamples of the training data.

12 Discussion

Both models achieve strong performance (93% accuracy, 0.97 ROC-AUC) on the Spambase dataset, confirming that the hand-engineered word/character-frequency features make spam detection close to linearly separable. Logistic Regression’s main strength here is its combination of top-tier accuracy, near-instant training time, and full interpretability via feature coefficients — a genuinely attractive package for a production email filter that also needs to be auditable. SVM’s RBF kernel matches it closely but at a noticeably higher hyperparameter-search cost and with less transparent decision-making. The Polynomial kernel’s poor baseline performance is a useful cautionary result: kernel choice without any tuning can catastrophically hurt recall, underscoring why hyperparameter search (Section 9) is not optional for kernel methods. A limitation of this study is that only accuracy was used as the tuning objective; for a real spam filter, a cost-sensitive metric (e.g., weighting false positives more heavily, since losing a legitimate email is often worse than one spam email slipping through) would likely change the selected threshold and possibly the selected hyperparameters. Future improvements could include probability-calibrated threshold tuning, ensembling the two models, and testing on a more recent, larger spam corpus to verify these features still generalize to modern spam patterns.

13 Conclusion

Logistic Regression and SVM were trained and tuned on the Spambase dataset to classify emails as spam or ham. After hyperparameter tuning (Grid Search for Logistic Regression, Randomized Search for SVM), Logistic Regression achieved the best held-out test accuracy (93.71%, F1 0.9203, ROC-AUC 0.9774) with an RBF-kernel SVM close behind (93.35% accuracy, F1 0.9154, ROC-AUC 0.9776). L1-regularized Logistic Regression was selected as best, indicating a sparse, interpretable subset of features drives most predictive power, and the near-equal performance of the Linear and RBF SVM kernels confirms the two classes are close to linearly separable in this feature space. Given its combination of accuracy, speed, and interpretability, Logistic Regression is the more practical choice for this dataset, while SVM remains a competitive, if costlier, alternative.

14 References

- 1 F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- 2 Scikit-learn Developers, “Logistic Regression,” *scikit-learn documentation*. [Online]. Available: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

- 3 Scikit-learn Developers, “Support Vector Machines,” *scikit-learn documentation*. [Online]. Available: <https://scikit-learn.org/stable/modules/svm.html>
- 4 Scikit-learn Developers, “Tuning the hyper-parameters of an estimator,” *scikit-learn documentation*. [Online]. Available: https://scikit-learn.org/stable/modules/grid_search.html
- 5 M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Dataset,” UCI Machine Learning Repository, 1999. [Online]. Available: <https://archive.ics.uci.edu/dataset/94/spambase>
- 6 “Spambase Dataset,” Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/somesh24/spambase>